DSC5001 Project Appendix

Source Codes

Hong Kong AQHI High-Risk Hour Early Warning

A Comparative Study of Classification Models on Hourly Monitoring Data

Team-XX

Contents:

1. Step 1: Data Download (01_download_data.py)
2. Step 2: Data Processing & Feature Engineering (02_feature_engineering.py)
3. Step 3: Model Training & Evaluation (03_model_training.py)
4. Step 4: Visualization Generation (04_visualizations.py)
5. Step 5: Slide Generation (05_generate_slides.py)

Step 1: Data Download

File: 01_download_data.py

```
"""
DSC5001 Project: Hong Kong AQHI High-Risk Hour Early Warning
============================================================
Step 1: Download AQHI hourly data and HKO daily weather data

Data sources:
- AQHI: https://www.aqhi.gov.hk/epd/ddata/html/history/YYYY/YYYYMM_Eng.csv
- Weather: https://data.weather.gov.hk/weatherAPI/cis/csvfile/HKO/ALL/daily_HKO_{PARAM}_ALL.csv
"""

import urllib.request
import os
import time
import sys

RAW_DIR = os.path.join(os.path.dirname(__file__), '..', 'raw_data')
os.makedirs(RAW_DIR, exist_ok=True)

# AQHI data: download 2021-2024 (4 years, avoiding 2025 AQHI methodology change)
AQHI_YEARS = [2021, 2022, 2023, 2024]
AQHI_MONTHS = list(range(1, 13))

# Weather data: HKO daily climate parameters
WEATHER_PARAMS = {
    'TEMP': 'daily_HKO_TEMP_ALL.csv',      # Daily Mean Temperature
    'RH':   'daily_HKO_RH_ALL.csv',        # Daily Mean Relative Humidity
    'RF':   'daily_HKO_RF_ALL.csv',        # Daily Total Rainfall
    'WET':  'daily_HKO_WET_ALL.csv',       # Daily Mean Wet Bulb Temperature
}

HEADERS = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'}

def download_file(url, filepath, retries=3):
    """Download a file with retry logic."""
    for attempt in range(retries):
        try:
            req = urllib.request.Request(url, headers=HEADERS)
            with urllib.request.urlopen(req, timeout=30) as resp:
                data = resp.read()
            with open(filepath, 'wb') as f:
                f.write(data)
            return True
        except Exception as e:
            if attempt &lt; retries - 1:
                time.sleep(2)
            else:
                print(f"  FAILED: {url} - {e}")
                return False
    return False


def download_aqhi_data():
    """Download monthly AQHI CSV files."""
    print("=" * 60)
    print("Downloading AQHI hourly data (2021-2024)...")


    print("=" * 60)
    success_count = 0
    fail_count = 0

    for year in AQHI_YEARS:
        for month in AQHI_MONTHS:
            filename = f"{year}{month:02d}_Eng.csv"
            filepath = os.path.join(RAW_DIR, filename)

            if os.path.exists(filepath) and os.path.getsize(filepath) &gt; 100:
                success_count += 1
                continue

            url = f"https://www.aqhi.gov.hk/epd/ddata/html/history/{year}/{filename}"
            print(f"  Downloading {filename}...", end=" ")
            sys.stdout.flush()

            if download_file(url, filepath):
```

```
                print("OK")
                success_count += 1
            else:
                fail_count += 1

            time.sleep(0.5)  # Be polite to the server

    print(f"\nAQHI download complete: {success_count} success, {fail_count} failed")
    return success_count, fail_count


def download_weather_data():
    """Download HKO daily climate data."""
    print("\n" + "=" * 60)
    print("Downloading HKO daily weather data...")
    print("=" * 60)
    success_count = 0

    for param, filename in WEATHER_PARAMS.items():
        filepath = os.path.join(RAW_DIR, filename)

        if os.path.exists(filepath) and os.path.getsize(filepath) &gt; 100:
            print(f"  {filename} - already exists")
            success_count += 1
            continue

        url = f"https://data.weather.gov.hk/weatherAPI/cis/csvfile/HKO/ALL/{filename}"
        print(f"  Downloading {filename}...", end=" ")
        sys.stdout.flush()

        if download_file(url, filepath):
            print("OK")
            success_count += 1
        else:
            print("FAILED")

    print(f"\nWeather download complete: {success_count}/{len(WEATHER_PARAMS)} success")


    return success_count


if __name__ == '__main__':
    aqhi_ok, aqhi_fail = download_aqhi_data()
    weather_ok = download_weather_data()

    print("\n" + "=" * 60)
    print("Data download summary:")
    print(f"  AQHI files: {aqhi_ok} downloaded, {aqhi_fail} failed")
    print(f"  Weather files: {weather_ok} downloaded")
    print(f"  Raw data directory: {os.path.abspath(RAW_DIR)}")
    print("=" * 60)
```

Step 2: Data Processing & Feature Engineering

File: 02_feature_engineering.py

```
"""
DSC5001 Project: Hong Kong AQHI High-Risk Hour Early Warning
============================================================
Step 2: Data processing, feature engineering, and target construction

- Parse AQHI hourly CSV files (2021-2024)
- Merge with HKO daily weather data
- Construct lagged features, time encodings
- Create binary target: next-hour AQHI &gt;= 7
"""

import os
import glob
import re
import numpy as np
import pandas as pd
from datetime import datetime

RAW_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'raw_data'))
DATA_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'data'))
os.makedirs(DATA_DIR, exist_ok=True)

# Station type mapping
ROADSIDE_STATIONS = ['Causeway Bay', 'Central', 'Mong Kok']
GENERAL_STATIONS = [
    'Central/Western', 'Southern', 'Eastern', 'Kwun Tong', 'Sham Shui Po',
    'Kwai Chung', 'Tsuen Wan', 'Tseung Kwan O', 'Yuen Long', 'Tuen Mun',
    'Tung Chung', 'Tai Po', 'Sha Tin', 'North', 'Tap Mun'
]

# AQHI risk threshold
AQHI_HIGH_RISK_THRESHOLD = 7


def parse_aqhi_file(filepath):
    """Parse a single AQHI monthly CSV file."""
    with open(filepath, 'r', encoding='utf-8-sig') as f:
        lines = f.readlines()

    # Find header line (starts with "Date,Hour")
    header_idx = None
    for i, line in enumerate(lines):
        if line.strip().startswith('Date,Hour'):
            header_idx = i
            break

    if header_idx is None:
        return pd.DataFrame()

    # Read the data
    headers = lines[header_idx].strip().split(',')
    data_lines = lines[header_idx + 1:]

    records = []
    current_date = None


    for line in data_lines:
        line = line.strip()
        if not line:
            continue
        parts = line.split(',')

        # Date might be empty (continuation from previous row)
        if parts[0].strip():
            current_date = parts[0].strip()

        if current_date is None:
            continue

        hour_str = parts[1].strip() if len(parts) &gt; 1 else ''
        if not hour_str:
            continue

        try:
```

```
            hour = int(hour_str)
        except ValueError:
            continue

        # Parse station values
        record = {'Date': current_date, 'Hour': hour}
        for j, station in enumerate(headers[2:]):
            if j + 2 &lt; len(parts):
                val = parts[j + 2].strip()
                # Remove asterisks (data from similar station)
                val = val.replace('*', '')
                try:
                    record[station] = int(val) if val else np.nan
                except ValueError:
                    record[station] = np.nan
            else:
                record[station] = np.nan

        records.append(record)

    df = pd.DataFrame(records)
    return df


def load_all_aqhi_data():
    """Load and combine all AQHI monthly files."""
    print("Loading AQHI data...")

    all_dfs = []
    aqhi_files = sorted(glob.glob(os.path.join(RAW_DIR, '*_Eng.csv')))

    # Filter to 2021-2024
    aqhi_files = [f for f in aqhi_files if re.match(r'.*202[1-4]\d{2}_Eng\.csv$', f)]

    print(f"  Found {len(aqhi_files)} monthly files")


    for filepath in aqhi_files:
        df = parse_aqhi_file(filepath)
        if not df.empty:
            all_dfs.append(df)

    combined = pd.concat(all_dfs, ignore_index=True)

    # Create datetime column
    combined['Date'] = pd.to_datetime(combined['Date'], format='%Y-%m-%d')
    combined['datetime'] = combined['Date'] + pd.to_timedelta(combined['Hour'], unit='h')

    # Sort
    combined = combined.sort_values('datetime').reset_index(drop=True)

    print(f"  Combined shape: {combined.shape}")
    print(f"  Date range: {combined['datetime'].min()} to {combined['datetime'].max()}")

    return combined


def reshape_to_long_format(df):
    """Reshape from wide (stations as columns) to long format."""
    print("Reshaping to long format...")

    station_columns = ROADSIDE_STATIONS + GENERAL_STATIONS
    available_stations = [s for s in station_columns if s in df.columns]

    long_df = df.melt(
        id_vars=['datetime', 'Date', 'Hour'],
        value_vars=available_stations,
        var_name='Station',
        value_name='AQHI'
    )

    # Drop rows with missing AQHI
    long_df = long_df.dropna(subset=['AQHI'])
    long_df['AQHI'] = long_df['AQHI'].astype(int)

    # Add station type
    long_df['StationType'] = long_df['Station'].apply(
        lambda x: 'Roadside' if x in ROADSIDE_STATIONS else 'General'
    )

    print(f"  Long format shape: {long_df.shape}")
```

```
    print(f"  Stations: {long_df['Station'].nunique()}")
    print(f"  Station types: {long_df['StationType'].value_counts().to_dict()}")

    return long_df


def load_weather_data():
    """Load and merge HKO daily weather data."""
    print("\nLoading weather data...")

    weather_dfs = {}


    def parse_hko_csv(filepath, param_name):
        """Parse HKO daily climate CSV, handling footer rows."""
        df = pd.read_csv(filepath, skiprows=2, encoding='utf-8-sig')
        # Keep only rows where Year column is numeric
        df = df[pd.to_numeric(df.iloc[:, 0], errors='coerce').notna()].copy()
        df.columns = ['Year', 'Month', 'Day', 'Value', 'Completeness']
        df['Year'] = df['Year'].astype(int)
        df['Month'] = df['Month'].astype(int)
        df['Day'] = df['Day'].astype(int)
        df['Value'] = pd.to_numeric(df['Value'].replace('***', np.nan), errors='coerce')
        # Build date string manually to avoid 1900-02-29 issues
        df = df[(df['Year'] &gt;= 1884) &amp; (df['Month'] &gt;= 1) &amp; (df['Month'] &lt;= 12) &amp; (df['Day'] &gt;= 1) &amp;
        df['Date'] = pd.to_datetime(df['Year'].astype(str) + '-' + df['Month'].astype(str).str.zfill(2) + '-' + df['Day'].astype
        df = df.dropna(subset=['Date'])
        result = df[['Date', 'Value']].dropna()
        result.columns = ['Date', param_name]
        return result

    # Temperature
    temp_file = os.path.join(RAW_DIR, 'daily_HKO_TEMP_ALL.csv')
    if os.path.exists(temp_file):
        weather_dfs['Temp'] = parse_hko_csv(temp_file, 'Temp')
        print(f"  Temperature: {len(weather_dfs['Temp'])} records")

    # Relative Humidity
    rh_file = os.path.join(RAW_DIR, 'daily_HKO_RH_ALL.csv')
    if os.path.exists(rh_file):
        weather_dfs['RH'] = parse_hko_csv(rh_file, 'RH')
        print(f"  Humidity: {len(weather_dfs['RH'])} records")

    # Rainfall
    rf_file = os.path.join(RAW_DIR, 'daily_HKO_RF_ALL.csv')
    if os.path.exists(rf_file):
        weather_dfs['Rainfall'] = parse_hko_csv(rf_file, 'Rainfall')
        print(f"  Rainfall: {len(weather_dfs['Rainfall'])} records")

    # Wet Bulb Temperature
    wet_file = os.path.join(RAW_DIR, 'daily_HKO_WET_ALL.csv')
    if os.path.exists(wet_file):
        weather_dfs['WetBulb'] = parse_hko_csv(wet_file, 'WetBulb')
        print(f"  Wet Bulb: {len(weather_dfs['WetBulb'])} records")

    # Merge all weather data
    weather = None
    for param, df in weather_dfs.items():
        if weather is None:
            weather = df
        else:
            weather = weather.merge(df, on='Date', how='outer')

    print(f"  Merged weather shape: {weather.shape}")
    return weather


def engineer_features(df, weather):
    """Create all features for the classification task."""
    print("\nEngineering features...")

    # Ensure datetime is sorted per station
    df = df.sort_values(['Station', 'datetime']).reset_index(drop=True)

    # --- Time features ---
    df['hour'] = df['datetime'].dt.hour
    df['dayofweek'] = df['datetime'].dt.dayofweek
    df['month'] = df['datetime'].dt.month
    df['dayofyear'] = df['datetime'].dt.dayofyear

    # Cyclical encoding of time features
```

```
    df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
    df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
    df['dow_sin'] = np.sin(2 * np.pi * df['dayofweek'] / 7)
    df['dow_cos'] = np.cos(2 * np.pi * df['dayofweek'] / 7)
    df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
    df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)

    # Is weekend
    df['is_weekend'] = (df['dayofweek'] &gt;= 5).astype(int)

    # --- Lagged AQHI features ---
    for lag in [1, 2, 3, 6, 12, 24]:
        df[f'AQHI_lag_{lag}'] = df.groupby('Station')['AQHI'].shift(lag)

    # --- Rolling statistics ---
    df['AQHI_rolling_mean_3'] = df.groupby('Station')['AQHI'].transform(
        lambda x: x.shift(1).rolling(3, min_periods=1).mean()
    )
    df['AQHI_rolling_mean_6'] = df.groupby('Station')['AQHI'].transform(
        lambda x: x.shift(1).rolling(6, min_periods=1).mean()
    )
    df['AQHI_rolling_std_6'] = df.groupby('Station')['AQHI'].transform(
        lambda x: x.shift(1).rolling(6, min_periods=1).std()
    )

    # AQHI trend (difference)
    df['AQHI_diff_1'] = df['AQHI_lag_1'] - df['AQHI_lag_2']
    df['AQHI_diff_3'] = df['AQHI_lag_1'] - df['AQHI_lag_3']

    # --- Target variable: next-hour AQHI &gt;= 7 ---
    df['next_AQHI'] = df.groupby('Station')['AQHI'].shift(-1)
    df['target'] = (df['next_AQHI'] &gt;= AQHI_HIGH_RISK_THRESHOLD).astype(int)

    # --- Merge weather data (by date) ---
    df['Date'] = pd.to_datetime(df['datetime'].dt.date)
    df = df.merge(weather, on='Date', how='left', suffixes=('', '_weather'))

    # Drop rows with missing target (last hour per station) or missing critical features
    critical_cols = ['AQHI_lag_1', 'AQHI_lag_2', 'AQHI_lag_3', 'next_AQHI', 'Temp', 'RH']
    df = df.dropna(subset=critical_cols)


    # Fill any remaining NaN in rolling features
    df['AQHI_rolling_std_6'] = df['AQHI_rolling_std_6'].fillna(0)

    # Rainfall might have NaN - fill with 0
    if 'Rainfall' in df.columns:
        df['Rainfall'] = df['Rainfall'].fillna(0)
    if 'WetBulb' in df.columns:
        df['WetBulb'] = df['WetBulb'].fillna(df['WetBulb'].median())

    # Station type encoding
    df['is_roadside'] = (df['StationType'] == 'Roadside').astype(int)

    print(f"  Final dataset shape: {df.shape}")
    print(f"  Target distribution:\n{df['target'].value_counts()}")
    print(f"  High-risk rate: {df['target'].mean():.4f} ({df['target'].mean()*100:.2f}%)")

    return df


def main():
    # Step 1: Load AQHI data
    aqhi_wide = load_all_aqhi_data()

    # Step 2: Reshape to long format
    aqhi_long = reshape_to_long_format(aqhi_wide)

    # Step 3: Load weather data
    weather = load_weather_data()

    # Step 4: Feature engineering
    data = engineer_features(aqhi_long, weather)

    # Save processed data
    output_path = os.path.join(DATA_DIR, 'processed_data.csv')
    data.to_csv(output_path, index=False)
    print(f"\nProcessed data saved to: {output_path}")
    print(f"Shape: {data.shape}")

    # Print summary statistics
```

```
    print("\n" + "=" * 60)
    print("DATASET SUMMARY")
    print("=" * 60)
    print(f"Total samples: {len(data):,}")
    print(f"Stations: {data['Station'].nunique()}")
    print(f"  Roadside: {data[data['is_roadside']==1]['Station'].nunique()}")
    print(f"  General: {data[data['is_roadside']==0]['Station'].nunique()}")
    print(f"Date range: {data['datetime'].min()} to {data['datetime'].max()}")
    print(f"High-risk (AQHI&gt;=7) rate: {data['target'].mean()*100:.2f}%")
    print(f"\nAQHI statistics:")
    print(data['AQHI'].describe())
    print(f"\nFeature columns:")
    feature_cols = [c for c in data.columns if c not in
                    ['datetime', 'Date', 'Hour', 'Station', 'StationType', 'AQHI', 'next_AQHI', 'target']]
    print(f"  {len(feature_cols)} features: {feature_cols}")


    return data


if __name__ == '__main__':
    main()
```

Step 3: Model Training & Evaluation

File: 03_model_training.py

```
"""
DSC5001 Project: Hong Kong AQHI High-Risk Hour Early Warning
============================================================
Step 3: Model training, evaluation, and comparison

Models:
  1. Logistic Regression (baseline, L2 regularization)
  2. Random Forest
  3. XGBoost (gradient boosting)

Evaluation:
  - TimeSeriesSplit cross-validation
  - Recall, Precision, F1, PR-AUC
  - Confusion matrices
  - Threshold optimization
  - Cost-benefit analysis (missed alerts vs false alarms)
"""

import os
import json
import warnings
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    precision_recall_curve, average_precision_score,
    confusion_matrix, classification_report
)
import xgboost as xgb

warnings.filterwarnings('ignore')

DATA_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'data'))
OUTPUT_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'output'))
FIGURES_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'figures'))
os.makedirs(OUTPUT_DIR, exist_ok=True)
os.makedirs(FIGURES_DIR, exist_ok=True)

# Feature columns
FEATURE_COLS = [
    'hour', 'dayofweek', 'month', 'dayofyear',
    'hour_sin', 'hour_cos', 'dow_sin', 'dow_cos', 'month_sin', 'month_cos',
    'is_weekend',
    'AQHI_lag_1', 'AQHI_lag_2', 'AQHI_lag_3', 'AQHI_lag_6', 'AQHI_lag_12', 'AQHI_lag_24',
    'AQHI_rolling_mean_3', 'AQHI_rolling_mean_6', 'AQHI_rolling_std_6',
    'AQHI_diff_1', 'AQHI_diff_3',
    'Temp', 'RH', 'Rainfall', 'WetBulb',
    'is_roadside'
]


RANDOM_STATE = 42


def load_data():
    """Load processed data."""
    print("Loading processed data...")
    data = pd.read_csv(os.path.join(DATA_DIR, 'processed_data.csv'), parse_dates=['datetime'])
    print(f"  Shape: {data.shape}")
    print(f"  High-risk rate: {data['target'].mean()*100:.2f}%")

    # Sort by datetime to ensure temporal order
    data = data.sort_values('datetime').reset_index(drop=True)

    # Fill NaN in feature columns with forward fill then median
    for col in FEATURE_COLS:
        if data[col].isna().any():
            median_val = data[col].median()
            data[col] = data[col].fillna(median_val)
```

```
    print(f"  NaN remaining in features: {data[FEATURE_COLS].isna().sum().sum()}")

    return data


def prepare_train_test(data, test_size=0.2):
    """Split data temporally - last 20% as test set."""
    n = len(data)
    split_idx = int(n * (1 - test_size))

    train = data.iloc[:split_idx].copy()
    test = data.iloc[split_idx:].copy()

    print(f"\nTemporal split:")
    print(f"  Train: {len(train):,} samples ({train['datetime'].min()} to {train['datetime'].max()})")
    print(f"  Test:  {len(test):,} samples ({test['datetime'].min()} to {test['datetime'].max()})")
    print(f"  Train high-risk rate: {train['target'].mean()*100:.2f}%")
    print(f"  Test high-risk rate:  {test['target'].mean()*100:.2f}%")

    return train, test


def train_logistic_regression(X_train, y_train):
    """Train Logistic Regression with class weights."""
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

    model = LogisticRegression(
        class_weight='balanced',
        C=1.0,
        penalty='l2',
        solver='lbfgs',
        max_iter=1000,
        random_state=RANDOM_STATE
    )


    model.fit(X_train_scaled, y_train)

    return model, scaler


def train_random_forest(X_train, y_train):
    """Train Random Forest with class weights."""
    model = RandomForestClassifier(
        n_estimators=200,
        max_depth=15,
        min_samples_split=50,
        min_samples_leaf=20,
        class_weight='balanced',
        random_state=RANDOM_STATE,
        n_jobs=-1
    )
    model.fit(X_train, y_train)
    return model, None


def train_xgboost(X_train, y_train):
    """Train XGBoost with scale_pos_weight."""
    neg_count = (y_train == 0).sum()
    pos_count = (y_train == 1).sum()
    scale_pos_weight = neg_count / pos_count

    model = xgb.XGBClassifier(
        n_estimators=300,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        scale_pos_weight=scale_pos_weight,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        eval_metric='aucpr',
        early_stopping_rounds=20
    )
    model.fit(X_train, y_train, eval_set=[(X_train, y_train)], verbose=False)
    return model, None


def find_optimal_threshold(y_true, y_proba, min_recall=0.7):
    """Find optimal threshold that maximizes F1 while maintaining minimum recall."""
```

```
    precision, recall, thresholds = precision_recall_curve(y_true, y_proba)

    # Filter to thresholds where recall &gt;= min_recall
    valid_mask = recall[:-1] &gt;= min_recall

    if valid_mask.sum() == 0:
        # If no threshold meets min_recall, use the one with highest recall
        best_idx = np.argmax(recall[:-1])
        return thresholds[best_idx], precision[best_idx], recall[best_idx]

    # Among valid thresholds, find the one with highest F1


    valid_precisions = precision[:-1][valid_mask]
    valid_recalls = recall[:-1][valid_mask]
    valid_thresholds = thresholds[valid_mask]

    f1s = 2 * valid_precisions * valid_recalls / (valid_precisions + valid_recalls + 1e-10)
    best_idx = np.argmax(f1s)

    return valid_thresholds[best_idx], valid_precisions[best_idx], valid_recalls[best_idx]


def evaluate_model(model, scaler, X_test, y_test, model_name, threshold=0.5):
    """Evaluate a model on the test set."""
    if scaler is not None:
        X_test_scaled = scaler.transform(X_test)
        y_proba = model.predict_proba(X_test_scaled)[:, 1]
    else:
        y_proba = model.predict_proba(X_test)[:, 1]

    # Default threshold predictions
    y_pred_default = (y_proba &gt;= 0.5).astype(int)

    # Find optimal threshold
    opt_threshold, opt_precision, opt_recall = find_optimal_threshold(y_test, y_proba, min_recall=0.65)
    y_pred_opt = (y_proba &gt;= opt_threshold).astype(int)

    # Calculate metrics for both thresholds
    results = {
        'model': model_name,
        'default_threshold': {
            'threshold': 0.5,
            'precision': float(precision_score(y_test, y_pred_default, zero_division=0)),
            'recall': float(recall_score(y_test, y_pred_default, zero_division=0)),
            'f1': float(f1_score(y_test, y_pred_default, zero_division=0)),
            'pr_auc': float(average_precision_score(y_test, y_proba)),
            'confusion_matrix': confusion_matrix(y_test, y_pred_default).tolist(),
        },
        'optimal_threshold': {
            'threshold': float(opt_threshold),
            'precision': float(precision_score(y_test, y_pred_opt, zero_division=0)),
            'recall': float(recall_score(y_test, y_pred_opt, zero_division=0)),
            'f1': float(f1_score(y_test, y_pred_opt, zero_division=0)),
            'pr_auc': float(average_precision_score(y_test, y_proba)),
            'confusion_matrix': confusion_matrix(y_test, y_pred_opt).tolist(),
        },
        'y_proba': y_proba.tolist() if len(y_proba) &lt; 5000 else y_proba[:5000].tolist(),
        'y_test': y_test.tolist() if len(y_test) &lt; 5000 else y_test[:5000].tolist(),
    }

    return results


def run_cross_validation(data, n_splits=5):
    """Run TimeSeriesSplit cross-validation."""
    print("\n" + "=" * 60)
    print("Running TimeSeriesSplit Cross-Validation")


    print("=" * 60)

    X = data[FEATURE_COLS].values
    y = data['target'].values

    tscv = TimeSeriesSplit(n_splits=n_splits)

    cv_results = {
        'logistic_regression': [],
        'random_forest': [],
        'xgboost': []
```

```
    }

    for fold, (train_idx, val_idx) in enumerate(tscv.split(X)):
        print(f"\n--- Fold {fold+1}/{n_splits} ---")
        X_train, X_val = X[train_idx], X[val_idx]
        y_train, y_val = y[train_idx], y[val_idx]

        print(f"  Train: {len(X_train):,} (high-risk: {y_train.sum():,}, {y_train.mean()*100:.2f}%)")
        print(f"  Val:   {len(X_val):,} (high-risk: {y_val.sum():,}, {y_val.mean()*100:.2f}%)")

        # Logistic Regression
        lr_model, lr_scaler = train_logistic_regression(X_train, y_train)
        lr_results = evaluate_model(lr_model, lr_scaler, X_val, y_val, 'Logistic Regression')
        cv_results['logistic_regression'].append({
            'precision': lr_results['optimal_threshold']['precision'],
            'recall': lr_results['optimal_threshold']['recall'],
            'f1': lr_results['optimal_threshold']['f1'],
            'pr_auc': lr_results['optimal_threshold']['pr_auc'],
        })
        print(f"  LR:   P={lr_results['optimal_threshold']['precision']:.4f}  R={lr_results['optimal_threshold']['recall']:.4f}

        # Random Forest
        rf_model, _ = train_random_forest(X_train, y_train)
        rf_results = evaluate_model(rf_model, None, X_val, y_val, 'Random Forest')
        cv_results['random_forest'].append({
            'precision': rf_results['optimal_threshold']['precision'],
            'recall': rf_results['optimal_threshold']['recall'],
            'f1': rf_results['optimal_threshold']['f1'],
            'pr_auc': rf_results['optimal_threshold']['pr_auc'],
        })
        print(f"  RF:   P={rf_results['optimal_threshold']['precision']:.4f}  R={rf_results['optimal_threshold']['recall']:.4f}

        # XGBoost
        xgb_model, _ = train_xgboost(X_train, y_train)
        xgb_results = evaluate_model(xgb_model, None, X_val, y_val, 'XGBoost')
        cv_results['xgboost'].append({
            'precision': xgb_results['optimal_threshold']['precision'],
            'recall': xgb_results['optimal_threshold']['recall'],
            'f1': xgb_results['optimal_threshold']['f1'],
            'pr_auc': xgb_results['optimal_threshold']['pr_auc'],
        })
        print(f"  XGB:  P={xgb_results['optimal_threshold']['precision']:.4f}  R={xgb_results['optimal_threshold']['recall']:.4f

    # Summarize CV results


    print("\n" + "=" * 60)
    print("Cross-Validation Summary (mean +/- std)")
    print("=" * 60)

    cv_summary = {}
    for model_name, folds in cv_results.items():
        metrics = {}
        for metric in ['precision', 'recall', 'f1', 'pr_auc']:
            values = [f[metric] for f in folds]
            metrics[metric] = {
                'mean': float(np.mean(values)),
                'std': float(np.std(values)),
            }
        cv_summary[model_name] = metrics
        print(f"\n{model_name}:")
        for metric, vals in metrics.items():
            print(f"  {metric}: {vals['mean']:.4f} +/- {vals['std']:.4f}")

    return cv_summary, cv_results


def train_final_models(train, test):
    """Train final models on full training set and evaluate on test set."""
    print("\n" + "=" * 60)
    print("Training Final Models on Full Training Set")
    print("=" * 60)

    X_train = train[FEATURE_COLS].values
    y_train = train['target'].values
    X_test = test[FEATURE_COLS].values
    y_test = test['target'].values

    print(f"\nTraining set: {len(X_train):,} samples")
    print(f"Test set: {len(X_test):,} samples")
```

```
all_results = {}
trained_models = {}

# Logistic Regression
print("\nTraining Logistic Regression...")
lr_model, lr_scaler = train_logistic_regression(X_train, y_train)
lr_results = evaluate_model(lr_model, lr_scaler, X_test, y_test, 'Logistic Regression')
all_results['logistic_regression'] = lr_results
trained_models['logistic_regression'] = (lr_model, lr_scaler)

# Random Forest
print("Training Random Forest...")
rf_model, _ = train_random_forest(X_train, y_train)
rf_results = evaluate_model(rf_model, None, X_test, y_test, 'Random Forest')
all_results['random_forest'] = rf_results
trained_models['random_forest'] = (rf_model, None)

# XGBoost
print("Training XGBoost...")
xgb_model, _ = train_xgboost(X_train, y_train)


xgb_results = evaluate_model(xgb_model, None, X_test, y_test, 'XGBoost')
all_results['xgboost'] = xgb_results
trained_models['xgboost'] = (xgb_model, None)

# Print results
print("\n" + "=" * 60)
print("Test Set Results (Optimal Threshold)")
print("=" * 60)
for model_name, results in all_results.items():
    r = results['optimal_threshold']
    print(f"\n{model_name}:")
    print(f"  Threshold: {r['threshold']:.4f}")
    print(f"  Precision: {r['precision']:.4f}")
    print(f"  Recall:    {r['recall']:.4f}")
    print(f"  F1:        {r['f1']:.4f}")
    print(f"  PR-AUC:    {r['pr_auc']:.4f}")
    cm = np.array(r['confusion_matrix'])
    print(f"  Confusion Matrix:")
    print(f"    TN={cm[0][0]:&gt;8,}  FP={cm[0][1]:&gt;8,}")
    print(f"    FN={cm[1][0]:&gt;8,}  TP={cm[1][1]:&gt;8,}")

# Feature importance
print("\n" + "=" * 60)
print("Feature Importance")
print("=" * 60)

feature_importance = {}

# Random Forest feature importance
rf_importance = trained_models['random_forest'][0].feature_importances_
rf_imp_df = pd.DataFrame({'feature': FEATURE_COLS, 'importance': rf_importance})
rf_imp_df = rf_imp_df.sort_values('importance', ascending=False)
feature_importance['random_forest'] = rf_imp_df.to_dict('records')
print("\nRandom Forest - Top 10 Features:")
for _, row in rf_imp_df.head(10).iterrows():
    print(f"  {row['feature']:&gt;25s}: {row['importance']:.4f}")

# XGBoost feature importance
xgb_importance = trained_models['xgboost'][0].feature_importances_
xgb_imp_df = pd.DataFrame({'feature': FEATURE_COLS, 'importance': xgb_importance})
xgb_imp_df = xgb_imp_df.sort_values('importance', ascending=False)
feature_importance['xgboost'] = xgb_imp_df.to_dict('records')
print("\nXGBoost - Top 10 Features:")
for _, row in xgb_imp_df.head(10).iterrows():
    print(f"  {row['feature']:&gt;25s}: {row['importance']:.4f}")

# Logistic Regression coefficients
lr_model = trained_models['logistic_regression'][0]
lr_scaler = trained_models['logistic_regression'][1]
lr_coef = lr_model.coef_[0]
lr_coef_df = pd.DataFrame({'feature': FEATURE_COLS, 'coefficient': lr_coef})
lr_coef_df['abs_coefficient'] = lr_coef_df['coefficient'].abs()
lr_coef_df = lr_coef_df.sort_values('abs_coefficient', ascending=False)
feature_importance['logistic_regression'] = lr_coef_df.to_dict('records')
print("\nLogistic Regression - Top 10 |Coefficients|:")


for _, row in lr_coef_df.head(10).iterrows():
    print(f"  {row['feature']:&gt;25s}: {row['coefficient']:+.4f}")
```

```
    # Cost-benefit analysis
    print("\n" + "=" * 60)
    print("Cost-Benefit Analysis")
    print("=" * 60)
    print("Assumption: Missing a high-risk alert (FN) is 5x more costly")
    print("            than a false alarm (FP)")

    cost_fn = 5
    cost_fp = 1

    for model_name, results in all_results.items():
        cm = np.array(results['optimal_threshold']['confusion_matrix'])
        tn, fp, fn, tp = cm[0][0], cm[0][1], cm[1][0], cm[1][1]
        total_cost = fn * cost_fn + fp * cost_fp
        # Baseline: predict all as non-high-risk
        baseline_cost = (tp + fn) * cost_fn
        cost_reduction = (baseline_cost - total_cost) / baseline_cost * 100

        print(f"\n{model_name}:")
        print(f"  Missed alerts (FN): {fn:,}")
        print(f"  False alarms (FP):  {fp:,}")
        print(f"  Total cost:         {total_cost:,}")
        print(f"  Baseline cost:      {baseline_cost:,}")
        print(f"  Cost reduction:     {cost_reduction:.2f}%")

        results['cost_analysis'] = {
            'fn': int(fn), 'fp': int(fp),
            'total_cost': int(total_cost),
            'baseline_cost': int(baseline_cost),
            'cost_reduction_pct': float(cost_reduction),
        }

    return all_results, trained_models, feature_importance


def main():
    # Load data
    data = load_data()

    # Cross-validation
    cv_summary, cv_results = run_cross_validation(data, n_splits=5)

    # Train final models
    train, test = prepare_train_test(data, test_size=0.2)
    all_results, trained_models, feature_importance = train_final_models(train, test)

    # Save results
    results_output = {
        'cv_summary': cv_summary,
        'cv_results': cv_results,
        'test_results': {k: {kk: vv for kk, vv in v.items() if kk not in ['y_proba', 'y_test']}
                         for k, v in all_results.items()},


        'feature_importance': feature_importance,
        'feature_cols': FEATURE_COLS,
    }

    results_path = os.path.join(OUTPUT_DIR, 'model_results.json')
    with open(results_path, 'w') as f:
        json.dump(results_output, f, indent=2)
    print(f"\nResults saved to: {results_path}")

    # Save predictions for plotting
    predictions = {}
    for model_name, results in all_results.items():
        predictions[model_name] = {
            'y_proba': results['y_proba'],
            'y_test': results['y_test'],
        }
    pred_path = os.path.join(OUTPUT_DIR, 'predictions.json')
    with open(pred_path, 'w') as f:
        json.dump(predictions, f)
    print(f"Predictions saved to: {pred_path}")

    return all_results, cv_summary, feature_importance


if __name__ == '__main__':
    main()
```

Step 4: Visualization Generation

File: 04_visualizations.py

```
"""
DSC5001 Project: Hong Kong AQHI High-Risk Hour Early Warning
============================================================
Step 4: Generate all visualizations and analysis plots

Plots:
  1. AQHI distribution and class imbalance
  2. PR curves for all three models
  3. Confusion matrices (optimal threshold)
  4. CV metrics comparison (boxplot)
  5. Feature importance (RF &amp;amp; XGBoost)
  6. LR coefficients
  7. Cost-benefit analysis
  8. Hourly AQHI pattern by station type
"""

import os
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from matplotlib.gridspec import GridSpec
import seaborn as sns

# Style settings
sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 150
plt.rcParams['savefig.dpi'] = 150
plt.rcParams['font.size'] = 10
plt.rcParams['axes.titlesize'] = 13
plt.rcParams['axes.labelsize'] = 11

# Colors
COLORS = {
    'logistic_regression': '#2196F3',   # Blue
    'random_forest': '#4CAF50',          # Green
    'xgboost': '#FF9800',                # Orange
}
MODEL_LABELS = {
    'logistic_regression': 'Logistic Regression',
    'random_forest': 'Random Forest',
    'xgboost': 'XGBoost',
}

DATA_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'data'))
OUTPUT_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'output'))
FIGURES_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'figures'))
os.makedirs(FIGURES_DIR, exist_ok=True)


def load_results():
    with open(os.path.join(OUTPUT_DIR, 'model_results.json'), 'r') as f:


        return json.load(f)


def load_data():
    return pd.read_csv(os.path.join(DATA_DIR, 'processed_data.csv'), parse_dates=['datetime'])


# ============================================================
# Plot 1: AQHI Distribution &amp;amp; Class Imbalance
# ============================================================
def plot_aqhi_distribution(data):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Left: AQHI value distribution
    ax = axes[0]
    counts = data['AQHI'].value_counts().sort_index()
    colors = ['#66BB6A' if v &lt; 7 else '#EF5350' for v in counts.index]
    bars = ax.bar(counts.index, counts.values, color=colors, edgecolor='white', linewidth=0.5)
```

```
    ax.axvline(x=6.5, color='red', linestyle='--', linewidth=1.5, label='Threshold (AQHI=7)')
    ax.set_xlabel('AQHI Value')
    ax.set_ylabel('Count')
    ax.set_title('Distribution of Hourly AQHI Values')
    ax.legend()

    # Add percentage labels on bars
    total = counts.sum()
    for bar, v in zip(bars, counts.values):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + total*0.005,
                f'{v/total*100:.1f}%', ha='center', va='bottom', fontsize=7)

    # Right: Target class distribution
    ax = axes[1]
    target_counts = data['target'].value_counts()
    labels = ['Non-High-Risk\n(AQHI &lt; 7)', 'High-Risk\n(AQHI &gt;= 7)']
    colors_pie = ['#66BB6A', '#EF5350']
    wedges, texts, autotexts = ax.pie(
        target_counts.values, labels=labels, colors=colors_pie,
        autopct='%1.2f%%', startangle=90, textprops={'fontsize': 11}
    )
    ax.set_title('Target Class Distribution\n(Class Imbalance Visualization)')

    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '01_aqhi_distribution.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 01_aqhi_distribution.png")


# ============================================================
# Plot 2: PR Curves
# ============================================================
def plot_pr_curves(results):
    # Load predictions
    with open(os.path.join(OUTPUT_DIR, 'predictions.json'), 'r') as f:
        predictions = json.load(f)


    from sklearn.metrics import precision_recall_curve, average_precision_score

    fig, ax = plt.subplots(figsize=(8, 6))

    for model_key in ['logistic_regression', 'random_forest', 'xgboost']:
        y_test = np.array(predictions[model_key]['y_test'])
        y_proba = np.array(predictions[model_key]['y_proba'])

        precision, recall, thresholds = precision_recall_curve(y_test, y_proba)
        pr_auc = average_precision_score(y_test, y_proba)

        ax.plot(recall, precision, color=COLORS[model_key], linewidth=2,
                label=f'{MODEL_LABELS[model_key]} (PR-AUC={pr_auc:.4f})')

    # Baseline (random classifier)
    positive_rate = np.mean(predictions['logistic_regression']['y_test'])
    ax.axhline(y=positive_rate, color='gray', linestyle='--', linewidth=1,
               label=f'Random Baseline ({positive_rate:.4f})')

    ax.set_xlabel('Recall (Sensitivity)')
    ax.set_ylabel('Precision')
    ax.set_title('Precision-Recall Curves (Test Set)')
    ax.legend(loc='upper right', fontsize=9)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1.05)
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '02_pr_curves.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 02_pr_curves.png")


# ============================================================
# Plot 3: Confusion Matrices
# ============================================================
def plot_confusion_matrices(results):
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    for idx, model_key in enumerate(['logistic_regression', 'random_forest', 'xgboost']):
        ax = axes[idx]
        cm = np.array(results['test_results'][model_key]['optimal_threshold']['confusion_matrix'])
```

```
        # Normalize for display
        cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

        # Plot as heatmap
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                    xticklabels=['Non-High-Risk', 'High-Risk'],
                    yticklabels=['Non-High-Risk', 'High-Risk'],
                    cbar=False)

        # Add normalized values as secondary annotation
        for i in range(2):
            for j in range(2):


                ax.text(j + 0.5, i + 0.65, f'({cm_norm[i][j]*100:.1f}%)',
                        ha='center', va='center', fontsize=8, color='gray')

        threshold = results['test_results'][model_key]['optimal_threshold']['threshold']
        ax.set_title(f'{MODEL_LABELS[model_key]}\n(threshold={threshold:.3f})')
        ax.set_xlabel('Predicted')
        ax.set_ylabel('Actual')

    plt.suptitle('Confusion Matrices (Optimal Threshold)', fontsize=14, y=1.02)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '03_confusion_matrices.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 03_confusion_matrices.png")


# ============================================================
# Plot 4: CV Metrics Comparison (Boxplot)
# ============================================================
def plot_cv_comparison(results):
    cv_data = results['cv_results']

    metrics = ['precision', 'recall', 'f1', 'pr_auc']
    metric_labels = ['Precision', 'Recall', 'F1 Score', 'PR-AUC']

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    for idx, (metric, label) in enumerate(zip(metrics, metric_labels)):
        ax = axes[idx // 2][idx % 2]

        plot_data = []
        labels = []
        colors = []

        for model_key in ['logistic_regression', 'random_forest', 'xgboost']:
            values = [fold[metric] for fold in cv_data[model_key]]
            plot_data.append(values)
            labels.append(MODEL_LABELS[model_key])
            colors.append(COLORS[model_key])

        bp = ax.boxplot(plot_data, tick_labels=labels, patch_artist=True, widths=0.6)
        for patch, color in zip(bp['boxes'], colors):
            patch.set_facecolor(color)
            patch.set_alpha(0.7)

        # Overlay individual points
        for i, (values, color) in enumerate(zip(plot_data, colors)):
            x = np.random.normal(i + 1, 0.04, size=len(values))
            ax.scatter(x, values, color=color, alpha=0.8, s=30, zorder=3)

        ax.set_ylabel(label)
        ax.set_title(f'{label} (5-Fold TimeSeriesSplit)')
        ax.set_ylim(0, 1)
        ax.grid(True, alpha=0.3, axis='y')

        # Rotate x labels


        ax.tick_params(axis='x', rotation=15)

    plt.suptitle('Cross-Validation Metrics Comparison', fontsize=14, y=1.02)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '04_cv_comparison.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 04_cv_comparison.png")


# ============================================================
```

```
# Plot 5: Feature Importance (RF &amp;amp; XGBoost)
# ============================================================
def plot_feature_importance(results):
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    for idx, model_key in enumerate(['random_forest', 'xgboost']):
        ax = axes[idx]
        imp_data = results['feature_importance'][model_key]

        # Take top 12 features
        df = pd.DataFrame(imp_data[:12])

        # Clean feature names for display
        df['feature'] = df['feature'].str.replace('_', ' ').str.title()

        bars = ax.barh(df['feature'][::-1], df['importance'][::-1],
                       color=COLORS[model_key], alpha=0.8, edgecolor='white')
        ax.set_xlabel('Feature Importance')
        ax.set_title(f'{MODEL_LABELS[model_key]} - Top 12 Features')
        ax.grid(True, alpha=0.3, axis='x')

        # Add value labels
        for bar, val in zip(bars, df['importance'][::-1]):
            ax.text(bar.get_width() + 0.005, bar.get_y() + bar.get_height()/2,
                    f'{val:.4f}', va='center', fontsize=8)

    plt.suptitle('Feature Importance Comparison', fontsize=14, y=1.02)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '05_feature_importance.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 05_feature_importance.png")


# ============================================================
# Plot 6: Logistic Regression Coefficients
# ============================================================
def plot_lr_coefficients(results):
    fig, ax = plt.subplots(figsize=(10, 6))

    coef_data = results['feature_importance']['logistic_regression']
    df = pd.DataFrame(coef_data[:15])
    df['feature'] = df['feature'].str.replace('_', ' ').str.title()

    colors = ['#EF5350' if c &gt; 0 else '#42A5F5' for c in df['coefficient']]


    bars = ax.barh(df['feature'][::-1], df['coefficient'][::-1],
                   color=colors[::-1], alpha=0.8, edgecolor='white')
    ax.set_xlabel('Coefficient Value')
    ax.set_title('Logistic Regression - Top 15 Features by |Coefficient|')
    ax.axvline(x=0, color='black', linewidth=0.8)
    ax.grid(True, alpha=0.3, axis='x')

    # Add value labels
    for bar, val in zip(bars, df['coefficient'][::-1]):
        offset = 0.03 if val &gt; 0 else -0.03
        ha = 'left' if val &gt; 0 else 'right'
        ax.text(bar.get_width() + offset, bar.get_y() + bar.get_height()/2,
                f'{val:+.3f}', va='center', ha=ha, fontsize=8)

    # Legend
    from matplotlib.patches import Patch
    legend_elements = [Patch(facecolor='#EF5350', alpha=0.8, label='Positive (increases risk)'),
                       Patch(facecolor='#42A5F5', alpha=0.8, label='Negative (decreases risk)')]
    ax.legend(handles=legend_elements, loc='lower right')

    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '06_lr_coefficients.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 06_lr_coefficients.png")


# ============================================================
# Plot 7: Cost-Benefit Analysis
# ============================================================
def plot_cost_benefit(results):
    test_results = results['test_results']

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    # Left: FN vs FP comparison
```

```
    ax = axes[0]
    models = []
    fn_values = []
    fp_values = []
    tp_values = []

    for model_key in ['logistic_regression', 'random_forest', 'xgboost']:
        models.append(MODEL_LABELS[model_key])
        cost = test_results[model_key]['cost_analysis']
        fn_values.append(cost['fn'])
        fp_values.append(cost['fp'])
        tp_values.append(test_results[model_key]['optimal_threshold']['confusion_matrix'][1][1])

    x = np.arange(len(models))
    width = 0.25

    bars1 = ax.bar(x - width, fn_values, width, label='Missed Alerts (FN)', color='#EF5350', alpha=0.8)
    bars2 = ax.bar(x, fp_values, width, label='False Alarms (FP)', color='#FFA726', alpha=0.8)
    bars3 = ax.bar(x + width, tp_values, width, label='Correct Alerts (TP)', color='#66BB6A', alpha=0.8)


    ax.set_xlabel('Model')
    ax.set_ylabel('Count')
    ax.set_title('Alert Outcomes on Test Set')
    ax.set_xticks(x)
    ax.set_xticklabels(models, rotation=15)
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')

    # Add value labels
    for bars in [bars1, bars2, bars3]:
        for bar in bars:
            ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                    f'{int(bar.get_height())}', ha='center', va='bottom', fontsize=8)

    # Right: Cost reduction
    ax = axes[1]
    cost_reductions = []
    total_costs = []
    baseline_costs = []

    for model_key in ['logistic_regression', 'random_forest', 'xgboost']:
        cost = test_results[model_key]['cost_analysis']
        cost_reductions.append(cost['cost_reduction_pct'])
        total_costs.append(cost['total_cost'])
        baseline_costs.append(cost['baseline_cost'])

    x = np.arange(len(models))
    width = 0.35

    bars1 = ax.bar(x - width/2, baseline_costs, width, label='Baseline Cost (No Model)', color='#BDBDBD', alpha=0.8)
    bars2 = ax.bar(x + width/2, total_costs, width, label='Model Cost', color='#42A5F5', alpha=0.8)

    ax.set_xlabel('Model')
    ax.set_ylabel('Total Cost (FN=5, FP=1)')
    ax.set_title('Cost-Benefit Analysis\n(Missing high-risk alert = 5x cost of false alarm)')
    ax.set_xticks(x)
    ax.set_xticklabels(models, rotation=15)
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')

    # Add cost reduction percentage annotations
    for i, reduction in enumerate(cost_reductions):
        ax.annotate(f'-{reduction:.1f}%', xy=(i, max(baseline_costs[i], total_costs[i]) + 100),
                    ha='center', fontsize=10, fontweight='bold', color='#2E7D32')

    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '07_cost_benefit.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 07_cost_benefit.png")


# ============================================================
# Plot 8: Hourly AQHI Pattern by Station Type
# ============================================================
def plot_hourly_pattern(data):


    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Left: Average AQHI by hour and station type
```

```
    ax = axes[0]
    hourly_avg = data.groupby(['hour', 'is_roadside'])['AQHI'].mean().unstack()

    ax.plot(hourly_avg.index, hourly_avg[1], color='#EF5350', linewidth=2, marker='o', markersize=4, label='Roadside')
    ax.plot(hourly_avg.index, hourly_avg[0], color='#42A5F5', linewidth=2, marker='s', markersize=4, label='General')
    ax.axhline(y=7, color='red', linestyle='--', linewidth=1, alpha=0.5, label='High-Risk Threshold (7)')
    ax.set_xlabel('Hour of Day')
    ax.set_ylabel('Average AQHI')
    ax.set_title('Average AQHI by Hour and Station Type')
    ax.set_xticks(range(0, 24, 2))
    ax.legend()
    ax.grid(True, alpha=0.3)

    # Right: High-risk rate by hour
    ax = axes[1]
    hourly_risk = data.groupby(['hour', 'is_roadside'])['target'].mean().unstack() * 100

    ax.plot(hourly_risk.index, hourly_risk[1], color='#EF5350', linewidth=2, marker='o', markersize=4, label='Roadside')
    ax.plot(hourly_risk.index, hourly_risk[0], color='#42A5F5', linewidth=2, marker='s', markersize=4, label='General')
    ax.set_xlabel('Hour of Day')
    ax.set_ylabel('High-Risk Rate (%)')
    ax.set_title('High-Risk (AQHI&gt;=7) Rate by Hour and Station Type')
    ax.set_xticks(range(0, 24, 2))
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '08_hourly_pattern.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 08_hourly_pattern.png")


# ============================================================
# Plot 9: Monthly High-Risk Pattern
# ============================================================
def plot_monthly_pattern(data):
    fig, ax = plt.subplots(figsize=(10, 5))

    monthly_risk = data.groupby(['month', 'is_roadside'])['target'].mean().unstack() * 100
    monthly_count = data.groupby('month')['target'].sum()

    ax.bar(monthly_risk.index - 0.15, monthly_risk[0], width=0.3, color='#42A5F5', alpha=0.8, label='General Stations')
    ax.bar(monthly_risk.index + 0.15, monthly_risk[1], width=0.3, color='#EF5350', alpha=0.8, label='Roadside Stations')

    ax.set_xlabel('Month')
    ax.set_ylabel('High-Risk Rate (%)')
    ax.set_title('Monthly High-Risk (AQHI&gt;=7) Rate by Station Type')
    ax.set_xticks(range(1, 13))
    ax.set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')


    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '09_monthly_pattern.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 09_monthly_pattern.png")


# ============================================================
# Plot 10: Threshold Sensitivity Analysis
# ============================================================
def plot_threshold_sensitivity(results):
    with open(os.path.join(OUTPUT_DIR, 'predictions.json'), 'r') as f:
        predictions = json.load(f)

    from sklearn.metrics import precision_recall_curve

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    for idx, model_key in enumerate(['logistic_regression', 'random_forest', 'xgboost']):
        ax = axes[idx]
        y_test = np.array(predictions[model_key]['y_test'])
        y_proba = np.array(predictions[model_key]['y_proba'])

        precision, recall, thresholds = precision_recall_curve(y_test, y_proba)

        # Plot precision and recall vs threshold
        ax.plot(thresholds, precision[:-1], color='#2196F3', linewidth=2, label='Precision')
        ax.plot(thresholds, recall[:-1], color='#EF5350', linewidth=2, label='Recall')
```

```
        # Plot F1
        f1 = 2 * precision[:-1] * recall[:-1] / (precision[:-1] + recall[:-1] + 1e-10)
        ax.plot(thresholds, f1, color='#4CAF50', linewidth=2, linestyle='--', label='F1')

        # Mark optimal threshold
        opt_threshold = results['test_results'][model_key]['optimal_threshold']['threshold']
        ax.axvline(x=opt_threshold, color='black', linestyle=':', linewidth=1.5, label=f'Optimal ({opt_threshold:.3f})')

        ax.set_xlabel('Threshold')
        ax.set_ylabel('Score')
        ax.set_title(MODEL_LABELS[model_key])
        ax.legend(fontsize=8)
        ax.set_xlim(0, 1)
        ax.set_ylim(0, 1.05)
        ax.grid(True, alpha=0.3)

    plt.suptitle('Threshold Sensitivity Analysis', fontsize=14, y=1.02)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '10_threshold_sensitivity.png'), bbox_inches='tight')
    plt.close()
    print("  Saved: 10_threshold_sensitivity.png")


# ============================================================
# Main
# ============================================================


def main():
    print("=" * 60)
    print("Generating Visualizations")
    print("=" * 60)

    results = load_results()
    data = load_data()

    print("\nGenerating plots...")
    plot_aqhi_distribution(data)
    plot_pr_curves(results)
    plot_confusion_matrices(results)
    plot_cv_comparison(results)
    plot_feature_importance(results)
    plot_lr_coefficients(results)
    plot_cost_benefit(results)
    plot_hourly_pattern(data)
    plot_monthly_pattern(data)
    plot_threshold_sensitivity(results)

    print(f"\nAll figures saved to: {FIGURES_DIR}")
    print(f"Total: 10 figures")


if __name__ == '__main__':
    main()
```

Step 5: Slide Generation

File: 05_generate_slides.py

```
"""
DSC5001 Project: Generate 10-page PowerPoint slides
Title: Hong Kong AQHI High-Risk Hour Early Warning
"""

import os
from pptx import Presentation
from pptx.util import Inches, Pt, Emu
from pptx.dml.color import RGBColor
from pptx.enum.text import PP_ALIGN, MSO_ANCHOR
from pptx.enum.shapes import MSO_SHAPE

FIGURES_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'figures'))
OUTPUT_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'output'))

# Color scheme - Academic blue/white
COLOR_TITLE = RGBColor(0x1A, 0x23, 0x7E)
COLOR_ACCENT = RGBColor(0x0D, 0x47, 0xA1)
COLOR_TEXT = RGBColor(0x21, 0x21, 0x21)
COLOR_LIGHT_BG = RGBColor(0xF5, 0xF7, 0xFA)
COLOR_WHITE = RGBColor(0xFF, 0xFF, 0xFF)
COLOR_RED = RGBColor(0xD3, 0x2F, 0x2F)
COLOR_GREEN = RGBColor(0x2E, 0x7D, 0x32)
COLOR_BLUE = RGBColor(0x15, 0x65, 0xC0)

prs = Presentation()
prs.slide_width = Inches(13.333)
prs.slide_height = Inches(7.5)

SLIDE_W = prs.slide_width
SLIDE_H = prs.slide_height


def add_textbox(slide, left, top, width, height, text, font_size=14, bold=False,
                color=COLOR_TEXT, alignment=PP_ALIGN.LEFT, font_name='Calibri'):
    txBox = slide.shapes.add_textbox(left, top, width, height)
    tf = txBox.text_frame
    tf.word_wrap = True
    p = tf.paragraphs[0]
    p.text = text
    p.font.size = Pt(font_size)
    p.font.bold = bold
    p.font.color.rgb = color
    p.font.name = font_name
    p.alignment = alignment
    return txBox


def add_bullet_list(slide, left, top, width, height, items, font_size=13,
                    color=COLOR_TEXT, spacing=Pt(6)):
    txBox = slide.shapes.add_textbox(left, top, width, height)
    tf = txBox.text_frame
    tf.word_wrap = True

    for i, item in enumerate(items):


        if i == 0:
            p = tf.paragraphs[0]
        else:
            p = tf.add_paragraph()

        if isinstance(item, tuple):
            p.text = item[0]
            p.font.bold = item[1]
        else:
            p.text = item
            p.font.bold = False

        p.font.size = Pt(font_size)
        p.font.color.rgb = color
        p.font.name = 'Calibri'
        p.space_after = spacing
        p.level = 0
```

```
    return txBox


def add_title_bar(slide, title_text, subtitle_text=None):
    """Add a colored title bar at the top."""
    # Background bar
    shape = slide.shapes.add_shape(
        MSO_SHAPE.RECTANGLE, 0, 0, SLIDE_W, Inches(1.0)
    )
    shape.fill.solid()
    shape.fill.fore_color.rgb = COLOR_ACCENT
    shape.line.fill.background()

    # Title text
    add_textbox(slide, Inches(0.5), Inches(0.15), Inches(12), Inches(0.5),
                title_text, font_size=24, bold=True, color=COLOR_WHITE,
                alignment=PP_ALIGN.LEFT)

    if subtitle_text:
        add_textbox(slide, Inches(0.5), Inches(0.65), Inches(12), Inches(0.3),
                    subtitle_text, font_size=12, color=RGBColor(0xBB, 0xDE, 0xFB),
                    alignment=PP_ALIGN.LEFT)


def add_image(slide, image_path, left, top, width=None, height=None):
    if width and height:
        return slide.shapes.add_picture(image_path, left, top, width, height)
    elif width:
        return slide.shapes.add_picture(image_path, left, top, width=width)
    elif height:
        return slide.shapes.add_picture(image_path, left, top, height=height)
    else:
        return slide.shapes.add_picture(image_path, left, top)


def add_footer(slide, page_num, total=10):
    add_textbox(slide, Inches(0.5), Inches(7.1), Inches(6), Inches(0.3),


                "DSC5001 Project | Hong Kong AQHI High-Risk Hour Early Warning",
                font_size=9, color=RGBColor(0x75, 0x75, 0x75))
    add_textbox(slide, Inches(11), Inches(7.1), Inches(2), Inches(0.3),
                f"{page_num} / {total}",
                font_size=9, color=RGBColor(0x75, 0x75, 0x75), alignment=PP_ALIGN.RIGHT)


# ============================================================
# Slide 1: Title Page
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])  # Blank layout

# Full background
shape = slide.shapes.add_shape(MSO_SHAPE.RECTANGLE, 0, 0, SLIDE_W, SLIDE_H)
shape.fill.solid()
shape.fill.fore_color.rgb = COLOR_ACCENT
shape.line.fill.background()

# Top accent line
shape = slide.shapes.add_shape(MSO_SHAPE.RECTANGLE, 0, Inches(2.5), SLIDE_W, Inches(0.06))
shape.fill.solid()
shape.fill.fore_color.rgb = RGBColor(0x82, 0xB1, 0xFF)
shape.line.fill.background()

# Course name
add_textbox(slide, Inches(1), Inches(1.0), Inches(11), Inches(0.5),
            "DSC5001 Statistical Machine Learning", font_size=18,
            color=RGBColor(0xBB, 0xDE, 0xFB), alignment=PP_ALIGN.CENTER)

# Project title
add_textbox(slide, Inches(1), Inches(2.8), Inches(11), Inches(1.0),
            "Hong Kong AQHI High-Risk Hour Early Warning:", font_size=28,
            bold=True, color=COLOR_WHITE, alignment=PP_ALIGN.CENTER)

add_textbox(slide, Inches(1), Inches(3.8), Inches(11), Inches(0.6),
            "A Comparative Study of Classification Models on Hourly Monitoring Data",
            font_size=18, color=RGBColor(0x90, 0xCA, 0xF9), alignment=PP_ALIGN.CENTER)

# Team info
add_textbox(slide, Inches(1), Inches(5.2), Inches(11), Inches(0.4),
            "Team-XX", font_size=16, bold=True, color=COLOR_WHITE, alignment=PP_ALIGN.CENTER)
```

```
add_textbox(slide, Inches(1), Inches(5.6), Inches(11), Inches(0.4),
            "[Student Name 1] (UID)  |  [Student Name 2] (UID)  |  [Student Name 3] (UID)  |  [Student Name 4] (UID)",
            font_size=12, color=RGBColor(0xBB, 0xDE, 0xFB), alignment=PP_ALIGN.CENTER)

add_textbox(slide, Inches(1), Inches(6.2), Inches(11), Inches(0.4),
            "October 2025", font_size=14, color=RGBColor(0x90, 0xCA, 0xF9), alignment=PP_ALIGN.CENTER)


# ============================================================
# Slide 2: Research Question &amp;amp; Motivation
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Research Question &amp;amp; Motivation")


# Left column - Question
add_textbox(slide, Inches(0.5), Inches(1.3), Inches(6), Inches(0.4),
            "Core Question", font_size=16, bold=True, color=COLOR_ACCENT)

add_bullet_list(slide, Inches(0.5), Inches(1.8), Inches(6), Inches(3.5), [
    "Can we reliably predict whether next-hour AQHI will reach high-risk level (&gt;=7) using past observations?",
    "How do different classifiers trade off between catching high-risk hours vs. controlling false alarms?",
    "Three models compared: Logistic Regression, Random Forest, XGBoost",
    ("Why classification, not regression?", True),
    "Public health advice is risk-level-based, not numeric -&gt; classification matches real application",
], font_size=12)

# Right column - Motivation
add_textbox(slide, Inches(7), Inches(1.3), Inches(6), Inches(0.4),
            "Motivation", font_size=16, bold=True, color=COLOR_ACCENT)

add_bullet_list(slide, Inches(7), Inches(1.8), Inches(6), Inches(3.5), [
    "EPD publishes AQHI hourly but forecasts only 3x/day (00:30, 10:30, 16:30)",
    "Hourly-level early warning has room for improvement during rapid pollution spikes",
    "AQHI time series has temporal dependency + severe class imbalance (&lt;2% high-risk)",
    ("Scientific value: not chasing best accuracy, but analyzing model trade-offs", True),
    "Missed alerts (FN) vs. false alarms (FP) have different real-world costs",
], font_size=12)

add_footer(slide, 2)


# ============================================================
# Slide 3: Data Sources &amp;amp; Feature Engineering
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Data Sources &amp;amp; Feature Engineering")

# Left - Data
add_textbox(slide, Inches(0.5), Inches(1.3), Inches(6), Inches(0.4),
            "Data Sources", font_size=16, bold=True, color=COLOR_ACCENT)

add_bullet_list(slide, Inches(0.5), Inches(1.8), Inches(6), Inches(4), [
    ("AQHI Hourly Data (EPD / data.gov.hk)", True),
    "18 stations: 15 general + 3 roadside (Causeway Bay, Central, Mong Kok)",
    "Period: Jan 2021 - Dec 2024 (48 months, avoiding 2025 methodology change)",
    "Total: 617,799 samples after preprocessing",
    ("Weather Data (HKO / data.gov.hk)", True),
    "Daily mean temperature, relative humidity, rainfall, wet bulb temp",
    "Merged by date with AQHI records",
], font_size=11)

# Right - Features
add_textbox(slide, Inches(7), Inches(1.3), Inches(6), Inches(0.4),
            "Feature Engineering (27 features)", font_size=16, bold=True, color=COLOR_ACCENT)

add_bullet_list(slide, Inches(7), Inches(1.8), Inches(6), Inches(4), [
    ("Lagged AQHI: lag-1, 2, 3, 6, 12, 24 hours", True),
    ("Rolling stats: 3h/6h mean, 6h std", True),


    ("AQHI differences: 1-step and 3-step", True),
    ("Time encoding: hour, day-of-week, month (sin/cos)", True),
    ("Weather: temperature, humidity, rainfall, wet bulb", True),
    ("Station type: roadside vs. general (binary)", True),
    ("Target: next-hour AQHI &gt;= 7 -&gt; binary (1.60% positive)", True),
], font_size=11)

add_footer(slide, 3)

# ============================================================
```

```
# Slide 4: Methodology
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Methodology")

# Left - Models
add_textbox(slide, Inches(0.5), Inches(1.3), Inches(6), Inches(0.4),
            "Three Classification Models", font_size=16, bold=True, color=COLOR_ACCENT)

add_bullet_list(slide, Inches(0.5), Inches(1.8), Inches(6), Inches(3), [
    ("1. Logistic Regression (baseline)", True),
    "  L2 regularization, class_weight='balanced'",
    ("2. Random Forest", True),
    "  200 trees, max_depth=15, class_weight='balanced'",
    ("3. XGBoost (Gradient Boosting)", True),
    "  300 rounds, max_depth=6, scale_pos_weight",
], font_size=12)

# Right - Evaluation Strategy
add_textbox(slide, Inches(7), Inches(1.3), Inches(6), Inches(0.4),
            "Evaluation Strategy", font_size=16, bold=True, color=COLOR_ACCENT)

add_bullet_list(slide, Inches(7), Inches(1.8), Inches(6), Inches(4), [
    ("TimeSeriesSplit CV (5 folds)", True),
    "  Prevents future data leakage",
    "  Uses only past data to predict future",
    ("Class imbalance handling:", True),
    "  Threshold optimization via PR curve",
    "  No SMOTE (preserves temporal structure)",
    ("Key metrics:", True),
    "  Recall (catch high-risk hours)",
    "  Precision (control false alarms)",
    "  F1, PR-AUC, Confusion Matrix",
    ("Cost analysis: FN cost = 5x FP cost", True),
], font_size=11)

add_footer(slide, 4)


# ============================================================
# Slide 5: Data Exploration
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Data Exploration: AQHI Distribution &amp;amp; Patterns")


# Left image - distribution
img_path = os.path.join(FIGURES_DIR, '01_aqhi_distribution.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(0.3), Inches(1.2), width=Inches(6.5))

# Right image - hourly pattern
img_path = os.path.join(FIGURES_DIR, '08_hourly_pattern.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(6.9), Inches(1.2), width=Inches(6.2))

# Key findings at bottom
add_textbox(slide, Inches(0.5), Inches(6.3), Inches(12), Inches(0.6),
    "Key findings: AQHI is right-skewed with only 1.60% high-risk (&gt;=7). Roadside stations show consistently higher AQHI than
    "High-risk hours peak during afternoon (12:00-19:00) across both station types.",
    font_size=11, color=COLOR_TEXT)

add_footer(slide, 5)


# ============================================================
# Slide 6: Cross-Validation Results
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Cross-Validation Results (5-Fold TimeSeriesSplit)")

# Left image - CV comparison boxplot
img_path = os.path.join(FIGURES_DIR, '04_cv_comparison.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(0.3), Inches(1.2), width=Inches(7.5))

# Right - Summary table
add_textbox(slide, Inches(8.2), Inches(1.3), Inches(5), Inches(0.4),
            "CV Summary (mean +/- std)", font_size=14, bold=True, color=COLOR_ACCENT)

# Table data
```

```
table_data = [
    ["Model", "Prec.", "Recall", "F1", "PR-AUC"],
    ["Log. Reg.", "0.566\n+/-0.118", "0.686\n+/-0.051", "0.616\n+/-0.090", "0.679\n+/-0.101"],
    ["Rand. Forest", "0.580\n+/-0.111", "0.693\n+/-0.039", "0.627\n+/-0.080", "0.663\n+/-0.111"],
    ["XGBoost", "0.481\n+/-0.119", "0.663\n+/-0.015", "0.550\n+/-0.088", "0.580\n+/-0.104"],
]

rows = len(table_data)
cols = len(table_data[0])
table = slide.shapes.add_table(rows, cols, Inches(8.2), Inches(1.8), Inches(5), Inches(3)).table

for i, row_data in enumerate(table_data):
    for j, cell_text in enumerate(row_data):
        cell = table.cell(i, j)
        cell.text = cell_text
        for paragraph in cell.text_frame.paragraphs:
            paragraph.font.size = Pt(10) if i &gt; 0 else Pt(11)
            paragraph.font.bold = (i == 0)
            paragraph.alignment = PP_ALIGN.CENTER


            if i == 0:
                paragraph.font.color.rgb = COLOR_WHITE

        if i == 0:
            cell.fill.solid()
            cell.fill.fore_color.rgb = COLOR_ACCENT
        elif i % 2 == 0:
            cell.fill.solid()
            cell.fill.fore_color.rgb = COLOR_LIGHT_BG

add_textbox(slide, Inches(8.2), Inches(5.5), Inches(5), Inches(1),
    "Logistic Regression and Random Forest perform comparably. XGBoost shows lower precision but more stable recall across folds
    font_size=10, color=COLOR_TEXT)

add_footer(slide, 6)


# ============================================================
# Slide 7: PR Curves &amp;amp; Confusion Matrices
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Test Set Results: PR Curves &amp;amp; Confusion Matrices")

# Left - PR curves
img_path = os.path.join(FIGURES_DIR, '02_pr_curves.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(0.3), Inches(1.2), width=Inches(6.3))

# Right - Confusion matrices
img_path = os.path.join(FIGURES_DIR, '03_confusion_matrices.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(6.8), Inches(1.2), width=Inches(6.5))

add_footer(slide, 7)


# ============================================================
# Slide 8: Feature Importance
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Feature Importance Analysis")

# Top left - RF &amp;amp; XGBoost importance
img_path = os.path.join(FIGURES_DIR, '05_feature_importance.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(0.3), Inches(1.2), width=Inches(7))

# Bottom left - LR coefficients
img_path = os.path.join(FIGURES_DIR, '06_lr_coefficients.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(0.3), Inches(4.0), width=Inches(7))

# Right - Key findings
add_textbox(slide, Inches(7.8), Inches(1.3), Inches(5.2), Inches(0.4),
            "Key Findings", font_size=16, bold=True, color=COLOR_ACCENT)


add_bullet_list(slide, Inches(7.8), Inches(1.8), Inches(5.2), Inches(5), [
    ("AQHI_lag_1 dominates all models", True),
    "  Most recent observation is strongest predictor",
```

```
    ("Rolling mean (3h) is 2nd most important", True),
    "  Recent trend captures momentum",
    ("Rolling std (6h) matters for RF", True),
    "  Volatility signals impending risk",
    ("Hour-of-day patterns are significant", True),
    "  Afternoon hours (12-19) show higher risk",
    ("Temperature has positive coefficient in LR", True),
    "  Higher temp -&gt; more high-risk hours",
    ("Rainfall has negative coefficient", True),
    "  Rain washes pollutants from air",
], font_size=11)

add_footer(slide, 8)


# ============================================================
# Slide 9: Cost-Benefit Analysis &amp; Threshold
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Cost-Benefit Analysis &amp; Threshold Optimization")

# Left - Cost benefit
img_path = os.path.join(FIGURES_DIR, '07_cost_benefit.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(0.3), Inches(1.2), width=Inches(6.8))

# Right - Threshold sensitivity
img_path = os.path.join(FIGURES_DIR, '10_threshold_sensitivity.png')
if os.path.exists(img_path):
    add_image(slide, img_path, Inches(7.3), Inches(1.2), width=Inches(5.8))

# Bottom findings
add_textbox(slide, Inches(0.5), Inches(6.3), Inches(12.5), Inches(0.6),
    "Assumption: missing a high-risk alert (FN) costs 5x a false alarm (FP). Random Forest achieves the highest cost reduction (
    "with the fewest missed alerts (448) despite more false alarms. Optimal thresholds are model-specific: LR=0.978, RF=0.893, X
    font_size=10, color=COLOR_TEXT)

add_footer(slide, 9)


# ============================================================
# Slide 10: Conclusions &amp; Limitations
# ============================================================
slide = prs.slides.add_slide(prs.slide_layouts[6])
add_title_bar(slide, "Conclusions &amp; Limitations")

# Left - Conclusions
add_textbox(slide, Inches(0.5), Inches(1.3), Inches(6), Inches(0.4),
            "Conclusions", font_size=16, bold=True, color=COLOR_GREEN)

add_bullet_list(slide, Inches(0.5), Inches(1.8), Inches(6), Inches(4.5), [


    ("Classification framework is viable for AQHI early warning", True),
    "  Binary prediction matches public health advice structure",
    ("Random Forest: best recall (73.8%) &amp; cost reduction (61.3%)", True),
    "  Recommended when missing high-risk hours is most costly",
    ("Logistic Regression: best precision (60.8%) &amp; highest PR-AUC", True),
    "  Recommended when false alarms must be minimized",
    ("XGBoost: balanced but lower overall performance", True),
    "  May benefit from more hyperparameter tuning",
    ("Key predictor: AQHI_lag_1 (most recent observation)", True),
    "  Simple lagged features outperform meteorological variables",
], font_size=11)

# Right - Limitations &amp; Future Work
add_textbox(slide, Inches(7), Inches(1.3), Inches(6), Inches(0.4),
            "Limitations &amp; Future Work", font_size=16, bold=True, color=COLOR_RED)

add_bullet_list(slide, Inches(7), Inches(1.8), Inches(6), Inches(4.5), [
    ("Weather data is daily, not hourly", True),
    "  Limits temporal resolution of meteorological features",
    ("Station spatial mismatch", True),
    "  EPD stations and HKO weather stations don't co-locate",
    ("AQHI methodology changed in March 2025", True),
    "  Used 2021-2024 data to avoid distribution shift",
    ("Class imbalance is severe (1.60%)", True),
    "  Even with weighting, precision remains moderate",
    ("Future: hourly weather data, multi-station spatial models", True),
    "  Deep learning (LSTM/Transformer) for sequence prediction",
], font_size=11)
```

```
add_footer(slide, 10)


# ============================================================
# Save
# ============================================================
output_path = os.path.join(OUTPUT_DIR, 'DSC5001_Project_Slides.pptx')
prs.save(output_path)
print(f"PPTX saved to: {output_path}")
print(f"Total slides: {len(prs.slides)}")
```